

Cahier des charges de Débitdouille

Debitdouille est un système open source permettant de suivre en temps réel les paramètres de pulvérisation pour un agriculteur, dans son tracteur, sur son téléphone.

Boîtier "hardware"

Un boîtier est installé sur le pulvé, il est composé :

- d'un ESP32, qui envoie les données en Bluetooth.
- De paires de débitmètres, un gauche et un droit. Il y a par défaut 1 paire, mais on peut aller jusqu'à 4 paires.
- D'un capteur de pression
- d'un GNSS permettant de calculer la vitesse.

Ce boîtier est branché depuis la cabine du tracteur, en 12V, sur une prise 3 plots. Il possède un régulateur 5V pour faire fonctionner les capteurs.

Les débitmètres et le capteur de pression sont à étalonner, au moins à l'installation du système, et au mieux une fois par an.

Les données du boîtier installé sur le pulvé sont affichées sur le smartphone de l'agriculteur via connexion bluetooth. Il y a donc 3 types d'infos principales à afficher ;

- **Pression** de la cuve, grâce à un capteur de pression
- **Débit**. Il peut y avoir plusieurs débitmètres installés sur le pulvé. Ils viennent par paire : Débit gauche 1 et débit droit 1, Débit gauche 2 et débit droit 2, Débit gauche 3 et débit droit 3.
- La **vitesse** est également donnée avec un GPS car celle donnée par le tracteur n'est pas bonne.

L'application peut choisir de se connecter à un objet en bluetooth BLE pour recevoir les trames DATA toutes les secondes en format JSON dans le format suivant :

```
{"P":"1.24","V":"7.30","DG1":"2.34","DD1":"0.29","DG2":"0.87","DD2":"0.00","DG3":"2.04","DD3":"1.11","DG4":"2.04","DD4":"1.11"}
```

Application "software"

L'application doit être simple, intuitive, sans fioritures.

Technologie :

L'application est développée en Flutter. Elle est composée de plusieurs fichiers qui correspondent à une architecture classique d'un projet flutter (main, screens, widgets, etc)

Pages :

Je décris ici les pages. Le style est simple : tout est en fond noir, écrit en blanc, le plus gros possible.

- **Base :**

Ceci est commun à toutes les pages. C'est TOUJOURS affiché, sur les pages décrites ci-dessous !

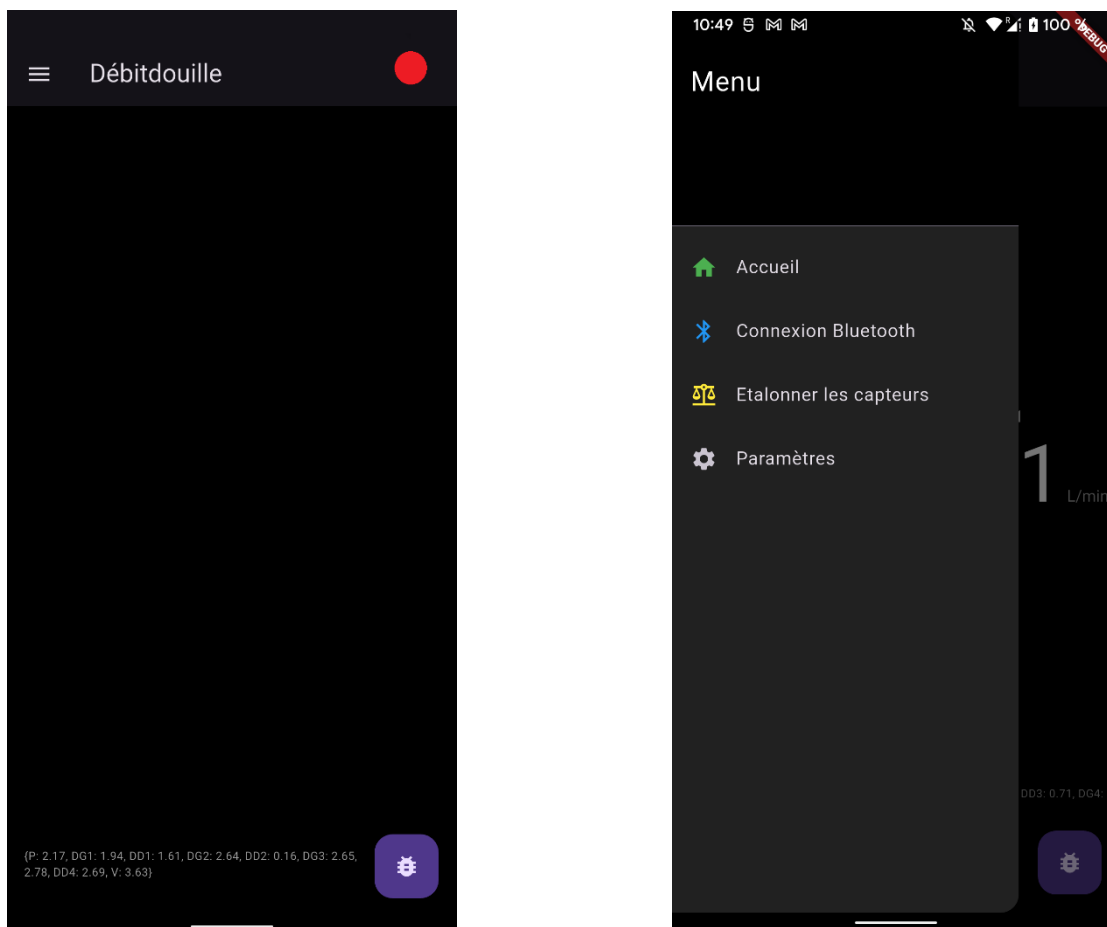
Il y a un en-tête avec un bouton « paramètres ». Ceci ouvre un onglet sur le côté, et on peut alors choisir entre les pages :

La page Accueil, la page de connexion, la page d'Etalonnage des capteurs, et la page de paramètres.

Sur l'en-tête, on trouve également un petit rond vert. Ce rond vire au rouge si aucune donnée n'a été reçue depuis 3secondes. Cela permet de s'assurer que le système est toujours connecté.

il y a un bouton en bas à droite permettant de simuler l'arrivée d'une trame json. Ce bouton peut être masqué, cela dépendra des options choisies dans la page des paramètres de l'appli. A gauche de ce bouton, tout en bas, on peut lire la dernière trame arrivée. Cela permet de déboguer. Cette trame json provient du bouton simulation OU du module Bluetooth.

L'exemple est donné ci-dessous :



A gauche, le scaffold avec les objets principaux. A droite, le menu avec les 4 pages.

- **page Accueil**

La page d'accueil de débitdouille est noir, avec écrit en blanc et en gros plusieurs chiffres :

- En haut, centré, la pression.
- Au milieu, les débits :
 - le (ou les) débits gauches à gauche
 - le (ou les) débits droits à droite
- En bas, centré, la vitesse.

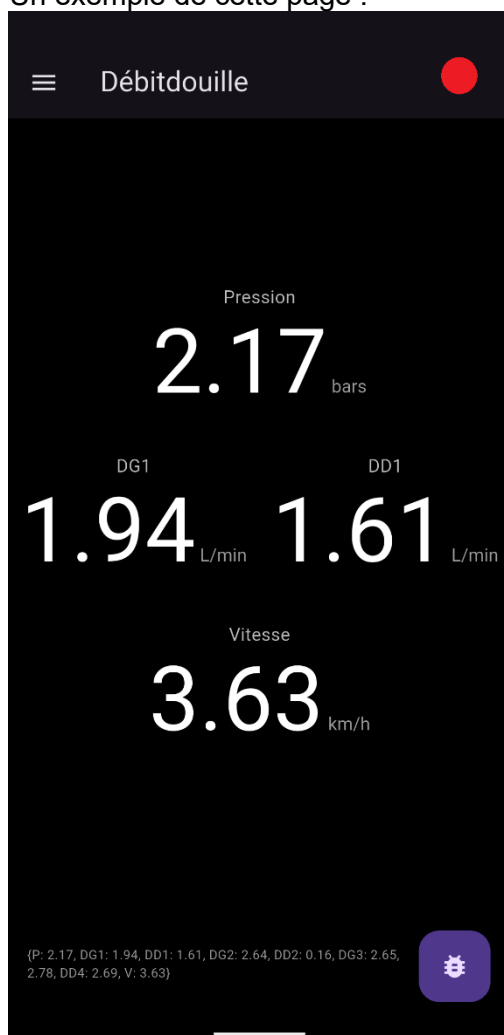
Comme c'est la page d'accueil, au début, toutes les valeurs sont à zéro.

Il est possible d'avoir plusieurs paires de débits gauche ou droit. Cela dépendra des options choisies dans la page des paramètres de l'appli.

Quoi qu'il arrive, l'affichage des débits est toujours au milieu de l'app, sur la gauche pour le débits gauche, et sur la droite pour les débit droits. Cela peut aller jusqu'à 4 débits gauche et 4 débits droit. La trame json contiendra toujours 4 paires de débits

Ces chiffres sont en gros. Au dessus de chaque chiffre, il est écrit en plus petit ce que c'est (pression, DG1, DG2, DD1, DD2, vitesse,). Il y a 2 décimales pour ces valeurs. Il y a aussi l'unité (bars, L/min ou km/h) affiché en petit, à droite du chiffre.

Un exemple de cette page :



Ci-dessus, la page d'accueil.

- **page Connexion**

La page de connexion permet d'appairer le boîtier. On peut lancer un scan et voir les appareils BT à proximité. Dès qu'une connexion est faite avec un module, on revient automatiquement sur la page principale, et on voit que les données sont mises à jour via le module Bluetooth. Si ça bugue, par exemple mauvais UUID, ça met un message d'erreur « connexion échouée »

- **page Paramètres**

La page des paramètres présente des boutons, qui permet de modifier l'interface :

- On peut choisir le nombre de paires de débits avec 1, 2, 3 ou 4 paires. Cela forcera à l'écran le nombre de paires de débits à afficher.
- Un bouton à cocher 'debug' permet de masquer ou afficher la trame json, affichée dans la page principale, faisant office de débog.
- Un bouton à cocher 'simulation' permet d'afficher sur la page principale un petit bouton, qui envoie une trame json fictive a chaque appui du bouton. Le bouton est masqué par défaut.

Si on rallume l'application, elle garde en mémoire les infos (il ne faut pas remodifier les réglages a chaque fois). Il n'y a pas de boutons « enregistrer » dans cette page, ça se fait automatiquement.

- **page Calibration**

Chaque capteur est calibré avec un coefficient A et B, de la façon $ValeurCalibrée = A * ValeurBrute + B$. Les trames JSON n'envoie que les valeurs calibrées.

La page de calibration permet de visualiser les coefficients du capteur, stocké sur l'ESP32, et de les corriger.

Chaque capteur est accessible avec une liste d'onglets en haut de l'écran, comme un carroussel de tous les capteurs (Pression, DG1, DG2, ...)

Dans la suite, je parle de l'onglet P (pression) mais les autres sont similaires.

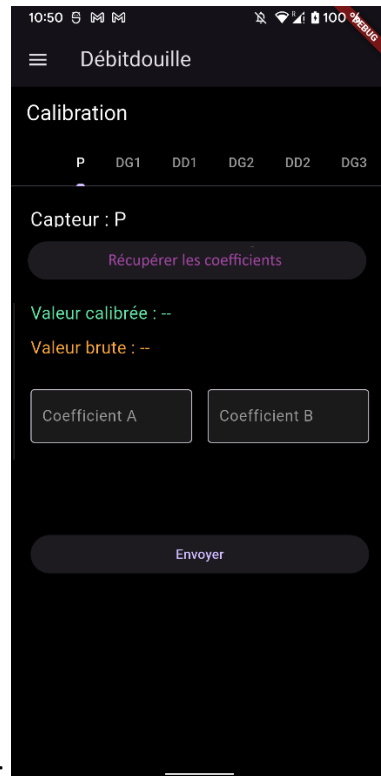
On a un bouton « récupérer les coefficients », qui permet d'afficher dans les CASES a ET b les coefficients stockées dans l'ESP32. La trame envoyée est de ce style, il suffira juste de filtrer pour le capteur voulu

```
{
"coeff": {
"DG1": {"A": 1.0, "B": 0.0},
"DD1": {"A": 1.0, "B": 0.0},
...
"P": {"A": 1.0, "B": 0.0}
}
```

On affiche alors : La Valeur brute (l'inverse de la formule mathématique ci-dessus), et la valeur calibrée.

On peut modifier manuellement les cases A et B pour changer de coefficient, cela modifiera la valeur calibrée affichée directement. Si on appuie sur « envoyer les coefficients », on va envoyer une trame json à l'esp32, qui va écraser les anciens coefficients et les mettre à jour. Cette trame json est du style :

```
{
"update_coeff": {
"DG1": {"A": 1.2, "B": -0.1},
...
}
```



Un exemple de page :